same tasks? Here's why:

- It is open source and free to use under the MIT licence.
- Julia is faster than Python and R. Yes, you read that right. Julia is by design faster at executing complex mathematical formulae as this was the original purpose behind creating the language.
- Julia supports concurrent parallel and distributed computing.
- Julia has the ability to directly call C or Fortran code without the requirement of additional glue code.
- Julia uses the 'Just Ahead of Time' (JAOT) compiler, which compiles the code to machine code by default before execution.
- Julia has some of the most efficient libraries for floating-point calculations and linear algebra (i.e., calculations involving matrices), which are essential for machine learning.
- Julia is supported by popular IDEs such as Visual Studio Code and execution environments such as Jupyter Notebook.
- It is super easy to install and get started.
- Julia has a vibrant online community that is active and increasing by the day.

Pre-requisites

Now that we have a pretty good grasp on what Julia is and what makes it ideal for machine learning, it's time to get our hands dirty. In this section of the article, we are going to implement a simple linear regression model to help us predict the prices of houses in Boston, Massachusetts, USA.

There are a few pre-requisites that need to be taken care of before getting started with the implementation:

- Visit *https://julialang.org/* and install the language in your OS. The procedure is pretty straightforward and Julia is available on all major platforms.
- Visit *https://jupyter.org/* and install Jupyter Notebook in your OS. Again, the procedure is pretty easy to follow, so we do not need to go into too many details.
- You will also need the standard Boston house prices data set used to demonstrate linear regression. Visit *https://www.kaggle.com/* to download.

Now that we have taken care of the necessary pre-requisites for this demonstration, let's get to the code.

Implementation

Developers familiar with Python are going to find many similarities in the syntax, structure and method of implementation in Julia. We will use a Jupyter Notebook for the compiling and execution of our code. So open up a fresh Jupyter Notebook in the desired folder and make sure to save your data set in the same location.

1. To start with, we are going to require certain libraries that we will make use of in this example. We will deal with data from a *csv* file. This will require us to use DataFrames. Additionally, we will perform some statistical calculations. Last but not the least, we will require a generalised linear model (GLM) for the implementation.

```
using DataFrames, CSV
using Plots
using GLM
using Statistics
using StatsPlots
```

2. Using the CSV and DataFrame libraries that we imported, we load the data from the data set.

```
# Read the file using CSV.File and convert it to DataFrame
df = DataFrame(CSV.File("Boston.csv"))
```

Out[2]: 5 rows × 15 columns (omitted printing of 5 columns)

	Column1	crim	zn	indus	chas	nox	rm	age	dis	rad
	Int64	Float64	Float64	Float64	Int64	Float64	Float64	Float64	Float64	Int64
1	1	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.09	1
2	2	0.02731	0.0	7.07	0	0.469	6.421	78.9	4.9671	2
3	3	0.02729	0.0	7.07	0	0.469	7.185	61.1	4.9671	2
4	4	0.03237	0.0	2.18	0	0.458	6.998	45.8	6.0622	3
5	5	0.06905	0.0	2.18	0	0.458	7.147	54.2	6.0622	3

Figure 1: The first five rows of the data set